

Optimized Point Addition Circuits for Elliptic Curve Discrete Logarithms

André Schrottenloher

Univ Rennes, Inria, CNRS, IRISA, Rennes, France
`firstname.lastname@inria.fr`

Abstract. Shor’s algorithm represents the main threat of quantum computers to cryptography. In order to precisely understand its feasibility, many authors have worked towards reducing its costs, either at the logical level (assuming a fault-tolerant architecture), or at the physical level (taking into account the constraints of envisioned hardware). In particular, recent works by Cheignard et al. (CRYPTO 2024) and Gidney (arXiv 2025) used improved arithmetic to significantly reduce the qubit cost of factoring RSA public keys.

Even more recently, Babbush et al. (arXiv 2026) improved the cost of computing elliptic curve discrete logarithms, with a reduction of a factor 2 to 3 in gate count and qubit count compared to a previous work by Litinski (arXiv 2023). Their result relies on optimized point addition circuits on elliptic curves over prime fields. However they did not reveal their logical quantum circuits, relying instead on a zero-knowledge proof. In this paper, we detail a quantum logical circuit architecture which gives similar results as Babbush et al., with a slightly higher number of qubits (around 1.5% increase) and a slightly smaller Toffoli gate count (between 6.5% and 10% reduction) for the curve secp256k1. We also give gate counts for a generic variant of the circuit, which is valid for any prime field.

Keywords: Quantum cryptanalysis, Shor’s algorithm, Discrete logarithms, Elliptic curves, Quantum resource estimates

1 Introduction

Since Shor’s breakthrough paper [21], it has been known that once a large-scale quantum computer becomes available, public-key cryptography based on the hardness of integer factoring (e.g., the RSA cryptosystem) and discrete logarithms (e.g., cryptography based on finite fields [16] and elliptic curve groups [16]) can be considered as broken, the underlying problems being solved in quantum polynomial time.

This impact motivates the early adoption of post-quantum cryptography, with a first batch of public-key encryption and signature schemes standardized by the NIST as a result of a large-scale public standardization process [18].

After Shor’s paper, many authors have quantified the *exact* gate and qubit counts of the algorithm, especially for its main cryptographic applications: factoring RSA integers, computing discrete logarithms in finite fields, and in elliptic curves of prime order. This paper focuses on the latter. Most of these studies [19,13,4] have considered logical quantum circuits, that is, without any implementation constraints. As the technology has matured, several works have also examined the cost of physically implementing the algorithm under specific assumptions regarding the capabilities and scale of the available hardware. Both logical algorithms and physical mappings have advanced in parallel. As an example, in 2021, Gidney and Ekerå [10] estimated that 20 million superconducting qubits could be necessary to factor RSA-2048 public keys (classically secure). A more recent paper by Gidney [9] reached one million qubits. Roughly half of this $20\times$ reduction (i.e., a factor 4) comes from logical circuits based on (and improving over) [3], while the rest comes from an improved physical implementation.

Babbush et al.’s Result. In a recent preprint, Babbush et al. [2] improved logical quantum circuits for computing elliptic curve discrete logarithms, and discussed the risks for cryptocurrencies using this non-post-quantum cryptography. In this paper, we only focus on their logical circuit results.

Babbush et al.’s result is based on an improved (windowed) elliptic curve point addition circuit on the curve `secp256k1`. Their circuits are “kickmick” circuits, which allow measurement-based uncomputation and phases but are ultimately efficient to simulate classically. They did not publish any detail on their circuit architecture, relying instead on a zero-knowledge proof to prove that they obtained circuits with the claimed gate and qubit counts, which performed point addition with a reasonable success probability. Succeeding on random inputs is enough to run Shor’s algorithm.

The decision not to publish the circuit is justified in [2] by concerns on the feasibility of such quantum attacks in practice on the medium term. This paper follows instead the practice of previous works [15,11,9], which intend to make the result fully reproducible. We believe that a full discussion of the arguments in [2] would be out of scope of the current paper.

Our Result. We obtain two circuits for point addition on `secp256k1`, reaching similar results as Babbush et al. [2], with a slight improvement in Toffoli gate count. The design principles of these circuits are described in this paper. We programmed and tested them using Qarton¹. In particular, the circuits are not exact: they will fail with some probability. Testing on random inputs shows that this probability of failure is comparable to the one of [2].

While the gate count varies depending on implementation details, the number of qubits is around $4.355n + \mathcal{O}(\sqrt{n})$ in the version that we selected, where n is the bit-size of the prime. It could be reduced up to $4.12n + \mathcal{O}(\sqrt{n})$ using a different implementation, which would likely increase the Toffoli gate count. Another

¹ <https://gitlab.inria.fr/capsule/qarton>

recent work [4] achieved a space of $3.12n + \mathcal{O}(\sqrt{n})$, but the technique employed is very different and yields a sharp increase in gate count for a relatively small gain in memory on relevant instances.

A comparison between the qubit and gate counts of Babbush et al. [2] and ours is given in Table 1.

Table 1. Costs of our circuits and comparison with [2]. The “Toffolis” column counts together CCX, CCZ as well as And gates. All circuits have a failure probability at most $2^{-13.3}$ (succeeding on 10 000 randomly chosen inputs). The number of qubits and gates given here do not account for the small window of size $w = 16$: it costs 16 qubits more and 3×2^w Toffoli gates to run the actual “windowed point addition” circuit.

Type	Qubits	Toffolis	Total gates	Reference
Space-optimized for secp256k1	1175	$2^{21.36}$	$\simeq 2^{24}$	[2]
Gate-optimized for secp256k1	1425	$2^{21.00}$	$\simeq 2^{24}$	[2]
Space-optimized for secp256k1	1192	$2^{21.19}$	$\simeq 2^{24}$	Ours
Gate-optimized for secp256k1	1446	$2^{20.83}$	$\simeq 2^{24}$	Ours
Space-optimized for any prime	1192	$2^{21.78}$	$\simeq 2^{24.5}$	Ours
Gate-optimized for any prime	1446	$2^{21.42}$	$\simeq 2^{24.5}$	Ours

Following the same reasoning as [2], the Toffoli count of the entire Shor’s algorithm on **secp256k1** is roughly:

$$28 \times (3 \times 2^{16} + Q_A) \quad (1)$$

where Q_A is the Toffoli count of a single windowed addition circuit as given in Table 1, while the qubit count is the one of Table 1 plus 16 (roughly). The algorithm needs to run only once to retrieve the discrete logarithm (contrary to trade-offs for RSA like in [9], which require multiple runs). The resulting numbers are summarized in Table 2. The best previous result was from Litinski [15] using around 200 million Toffoli gates (i.e., $2^{27.57}$) to compute a single discrete logarithm, with about twice as much qubits.

Ideas of the Circuit and Organization of the Paper

We detail our circuits in a top-down approach, starting from the high-level implementation of Shor’s algorithm (which is the same as in [2] and previous works), down to the implementation of arithmetic circuits.

From Shor’s Algorithm to Point Addition (Section 2). The fact that Babbush et al. [2] used a standard implementation of Shor’s algorithm, and not a “compressed” version like [4], is explicit in their work. In this implementation, the main costly component in the algorithm is point multiplication:

$$|u, v\rangle |0\rangle \mapsto |u, v\rangle |[u]P + [v]Q\rangle \quad ,$$

Table 2. Cost of running Shor’s algorithm entirely and comparison with [2]. The “Toffolis” column counts together CCX, CCZ as well as And gates.

Type	Qubits	Toffolis	Reference
Space-optimized for secp256k1	1191	$28 \times (2^{21.36} + 3 \cdot 2^{16}) = 2^{26.27}$	[2]
Gate-optimized for secp256k1	1441	$28 \times (2^{21.00} + 3 \cdot 2^{16}) = 2^{25.94}$	[2]
Space-optimized for secp256k1	1208	$28 \times (2^{21.19} + 3 \cdot 2^{16}) = 2^{26.11}$	Ours
Gate-optimized for secp256k1	1462	$28 \times (2^{20.83} + 3 \cdot 2^{16}) = 2^{25.78}$	Ours
Space-optimized for any prime	1208	$28 \times (2^{21.78} + 3 \cdot 2^{16}) = 2^{26.66}$	Ours
Gate-optimized for any prime	1462	$28 \times (2^{21.42} + 3 \cdot 2^{16}) = 2^{26.33}$	Ours

where P, Q are points on the elliptic curve given by the discrete logarithm instance (Q is the input and P is the basis of the discrete logarithm). Although the registers u, v take n bits each roughly, where n is the bit-size of the group of points, they do not count towards the total qubit count thanks to the *semi-classical Fourier transform* with qubit recycling, and we can focus instead on the points.

Point multiplication is then reduced to *windowed point addition*, where one uses a w -bit integer input i to select a point from a built-in *window* $(P_0, \dots, P_{2^w-1})$, and add the constant point to the variable input:

$$|i\rangle |R\rangle \mapsto |i\rangle |R + P_i\rangle .$$

Finally, the point addition by a constant is done by an algorithm of [11]. Points are represented using *affine* coordinates, i.e., a pair of integers modulo q . The addition by P_i is reduced to a sequence of arithmetic operations modulo q and three *lookups*, which are the circuits loading the coordinates of P_i from the input i . All this high-level structure is detailed in Section 2 and is explicit in Babbush et al.’s paper [2].

From In-Place Multiplication to the Extended Euclidean Algorithm (Section 3). When doing point addition like this, one observes that the most costly step is to *multiply in-place* two integers modulo q : $|x, y\rangle \mapsto |x, xy\rangle$. This operation happens twice in the point addition circuit. Previous works like [19, 13, 11, 15] reduced it to *modular inversion* and out-of-place multiplication ($|x, y, z\rangle \mapsto |x, y, z + xy\rangle$), and implemented modular inversion using reversible variants of binary Extended Euclidean Algorithm (EEA). While inversion and multiplication on n -bit modular integers have both a gate count in $\mathcal{O}(n^2)$ (as does the entire point addition circuit), the constant is noticeably bigger for the inversion part.

In Section 3, we detail the implementation of this step. The main idea, taken from [14], is to separate the EEA into two sub-circuits. The first circuit is an Euclidean algorithm which operates on a pair of integers (u, v) (which should start at (q, x) if we are inverting x modulo q), performs the operations of the Euclidean algorithm, and outputs a bit-string describing the operations that it has performed, ensuring reversibility. This sequence of operations is named a

dialog in [14], which used this for polynomials, because it can contain a division by 2, an inversion (swapping the integers), or an addition. The second circuit is a *Bézout reconstruction* algorithm which takes as input the sequence of operations from the Euclidean algorithm, as well as a pair (r, s) (which should start at $(1, 0)$ if we are inverting x modulo q), and applies the appropriate operations on (r, s) . Essentially, this two-step process is splitting the EEA between the input part and the Bézout coefficients. This has many advantages which are detailed in [Section 3](#): not only is the space reduced, but this point of view allows to directly implement the in-place multiplication, saving several modular multiplications.

The total space complexity of the point addition ends up being dominated by the Bézout reconstruction step. As we observe in [Section 3](#), in the most space-optimized implementation, the sequence of steps of the EEA can be stored on $2.12n + \mathcal{O}(\sqrt{n})$ qubits. During the Bézout reconstruction step, one needs to store in addition a pair of integers modulo q , of n bits each. Therefore the circuit uses at least $4.12n + \mathcal{O}(\sqrt{n})$ qubits. In our implementation, we use slightly more space than this, leading to our $4.36n + \mathcal{O}(\sqrt{n})$ asymptotic. One should note that the constant in the $\mathcal{O}$ controls the success probability of the algorithm on random inputs, and we optimize it following experimental results.

Arithmetic Circuits. At this point, the cost of the point addition circuit will depend on trade-offs in the circuits for unsigned arithmetic (in the Euclidean algorithm) and modular arithmetic (in the Bézout reconstruction and elsewhere). For unsigned arithmetic, we simply switch between the CDKM adder [5], the Gidney adder [7] and a hybrid between the two, depending on the number of ancilla qubits available. We also rely on Gidney’s low-space classical-quantum adder [8]. For modular arithmetic, we introduce two levels of optimization which, to the best of our knowledge, have not been described before, and could have applications in other quantum cryptanalysis circuits relying on modular arithmetic. Contrary to most recent works [13, 11, 15], we do not rely on the Montgomery representation of modular integers, instead optimizing the circuits as much as possible under a standard representation of integers. The components that we optimize are modular doubling and controlled modular addition, taken from [20].

We give a first batch of optimizations working for any modulus q , where comparisons are simplified by comparing the most significant bits only: the amount of MSBs determines the success probability of the circuit. Then, we observe that the prime in `secp256k1` is a *pseudo-Mersenne* prime, of the form $2^u - f$ where $f \ll 2^u$. This allows to further simplify the modular reductions which appear in these circuits. The difference between these two levels of optimization explains the difference between `secp256k1` and “any prime” reported in [Table 1](#).

Implementation. We provide a complete implementation of the circuits described in this paper using the python library Qarton. Our code is available at:

gitlab.inria.fr/capsule/qarton-projects/ec-point-addition

Using this code, one can for example build the point addition circuit, simulate it on random inputs and check that the error probability is small. We note that for efficiency reasons, the simulation does not descend at the gate level. Instead it replaces *exact* arithmetic circuits (addition, subtraction, comparison...) by their classical functions.

2 High-Level Structure

This section details the reduction from Shor’s algorithm for discrete logarithms to the *windowed point addition* circuit on elliptic curves, and gives the high-level structure of the circuit.

Let $E(\mathbb{F}_q)$ be an elliptic curve defined over a prime field, where q is a prime number of bit-size n ($q < 2^n$). The curve is defined by an equation $y^2 = x^3 + ax + b$. In particular, the `secp256k1` curve uses:

$$q = 2^{256} - 4294968273, a = 0, b = 7.$$

A point on the curve in *affine coordinates* is either of the form (x, y) , or the *point at infinity* $\mathcal{O}$. From an implementation perspective, these representations can be unified by a triple $(x, y, b) \in \mathbb{F}_q \times \mathbb{F}_q \times \{0, 1\}$ where $b = 0$ and $x = y = 0$ if the point is $\mathcal{O}$ and $b = 1$ otherwise. Points on the curve form an additive group where $\mathcal{O}$ is the neutral element.

Quantum Circuits. We describe quantum algorithms in the quantum circuit model. We refer to [17] for an overview, and assume familiarity with the notions of gates, qubits, and the ket notation $|\cdot\rangle$ of quantum states.

We focus on the optimization of *logical* quantum circuits, without taking into account the error correction layer of fault-tolerant quantum architectures. At the logical layer, we are interested in minimizing the number of logical qubits, and the gate count, especially the non-Clifford gates, due to the higher difficulty of implementing them in fault-tolerant quantum computing. In previous works, Toffoli gates or T gates have been used as the main metric. In this paper, we use circuits which contain Toffoli gates or AND gates, which are decomposed differently in the Clifford+T gate set. Both are counted together as a single “Toffoli gate” metric. These are the only non-Clifford gates in the circuits. Other gates include not (X), controlled not (CX), Hadamard, phase flips (CZ).

All arithmetic circuits described in this paper can be efficiently simulated, despite using specific quantum effects like measurement-based uncomputation. Babbush et al. [2] notice this for their circuits, and describe a class of “kickmix” circuits which operate using classical gates, phases and measurements. More generally, the property we require (which is satisfied throughout this paper) is that simulating the circuit, starting from any valid input (a single basis state), produces a quantum state with a sparse support, having always at most a constant number of basis states. With measurement-based uncomputation, like in [8], one typically applies Hadamard gates followed immediately by measurements, so the quantum state remains supported by two basis states only.

Shor's Algorithm. Let P be a base point and Q be the point of which we compute the discrete logarithm, i.e., $Q = [k]P$ for some unknown k . We use a version of Shor's algorithm that can be adapted with the semiclassical Fourier transform [12]:

1. Create a uniform superposition over two registers of size n : $\sum_{0 \leq u, v < 2^n} |u\rangle |v\rangle$
2. Apply a *point multiplication unitary*: $\sum_{0 \leq u, v < 2^n} |u\rangle |v\rangle |[u]P + [v]Q\rangle$
3. Apply a 2^n -dimensional Fourier transform over the two input registers
4. Measure

One can then retrieve k from a single measurement result with large success probability (more than $1/2$). The *semiclassical Fourier transform* interleaves the operations of the Fourier transform and the point multiplication using a single recycled control qubit. As a consequence, both qubit and gate count can be approximated by those of the point multiplication:

$$|u, v\rangle |0\rangle \mapsto |u, v\rangle |[u]P + [v]Q\rangle . \quad (2)$$

Let w be a *window* size, which can be selected depending on the cost of point addition. The value is $w = 16$ in [2], and since we have similar costs, we will use the same later on. We rewrite u and v as:

$$\begin{cases} u := u_0 + 2^w u_1 + 2^{2w} u_2 + \dots + 2^{(\ell-1)w} u_{\ell-1} \\ v := v_0 + 2^w v_1 + 2^{2w} v_2 + \dots + 2^{(\ell-1)w} v_{\ell-1} \end{cases} \quad (3)$$

Then the point multiplication is rewritten as:

$$[u]P + [v]Q = \sum_{i=0}^{\ell-1} [u_i]([2^{iw}]P) + \sum_{i=0}^{\ell-1} [v_i]([2^{iw}]Q) . \quad (4)$$

We can therefore reduce the point multiplication circuit to the following *windowed point addition*. Let $(P_0, \dots, P_{2^w-1})$ be a *window* of points, then the point addition circuit does:

$$|i\rangle |R\rangle \mapsto |i\rangle |R + P_i\rangle . \quad (5)$$

Starting from $\mathcal{O}$, we just have to compute $2\ell = (2n/w)$ times a windowed point addition circuit, where the windows are precomputed multiples of P and Q . If $i = 0$ the multiple is $\mathcal{O}$. The semiclassical Fourier transform works similarly for windowed additions, with less intermediate measurements since we only have to interleave the operations between each window.

Point Addition. The point addition circuit uses the strategy of [11]. We can use formulas for addition of elliptic curve points in affine coordinates, neglecting the case of doublings (which would happen only with negligible probability) and the point at infinity. The only special case is for $i = 0$, where the output register should be unchanged since we are adding $\mathcal{O}$.

Algorithm 1 Reversible point addition circuit (from [11], slightly modified to allow addition of $\mathcal{O}$).

Parameter: window of points $P_0, \dots, P_{2^w-1}$
Input: Registers x_2, y_2 (coordinates of R), register i
Output: Registers are modified in-place and contain $R + P_i$

- 1: $c \leftarrow (i \neq 0)$
- 2: $x_1, y_1 \leftarrow \text{Lookup}(P_i)$ ▷ First lookup
▷ When $i = 0$, we have $x_1, y_1 = 0$ per our definition
- 3: $x_2 \leftarrow x_2 - x_1$
- 4: $y_2 \leftarrow y_2 - y_1$
- 5: $\text{Unlookup}(P_i)$ ▷ Removes x_1, y_1
- 6: $x_2, y_2 \leftarrow x_2, y_2 \cdot x_2^{-1}$ ▷ Inverse of an in-place multiplication
- 7: $x \leftarrow \text{Lookup}(3x_{P_i})$ ▷ Second lookup
- 8: $x_2 \leftarrow x_2 + x$
- 9: $\text{Unlookup}(3x_{P_i})$ ▷ Removes x
- 10: If c then $x_2 \leftarrow x_2 - y_2^2$ ▷ Controlled modular squaring
- 11: $x_2, y_2 \leftarrow x_2, y_2 \cdot x_2$ ▷ In-place multiplication
- 12: If c then $x_2 \leftarrow -x_2$ ▷ Controlled modular negation
- 13: $x_1, y_1 \leftarrow \text{Lookup}(P_i)$ ▷ Third lookup
- 14: $y_2 \leftarrow y_2 - y_1$
- 15: $x_2 \leftarrow x_2 + x_1$
- 16: $\text{Unlookup}(P_i)$ ▷ Removes x_1, y_1
- 17: Uncompute c

During the computation of the formulas, we need to use the coordinates of P_i three separate times: first the entire point, then $3x_{P_i}$, then the entire point again. Each time, we load these coordinates using a *table lookup* circuit, and unload them immediately after use. A table lookup of 2^w values costs 2^w Toffoli gates [1], and the unlookup is almost free thanks to measurement-based uncomputation. In addition, the Toffoli gate count of lookups being still an order of magnitude below the cost of the other operations, we could also just compute the inverse of the lookup circuit.

As can be seen in Algorithm 1, the circuit contains 3 lookups, some arithmetic operations in $\mathcal{O}(n)$ gates, a modular squaring, and two *in-place multiplication* circuits which we will detail next.

Number of Point Additions. The number of point additions can slightly be optimized [2], following [15]. Indeed, the first addition can be replaced by a lookup of the result, and the last three ones can be removed at the cost of more classical post-processing [6]. In the `secp256k1` case, this means that only 28 point additions are necessary.

3 Modular In-Place Multiplication

The modular in-place multiplication:

$$|x\rangle |y\rangle \mapsto |x\rangle |yx\rangle \quad (6)$$

is the core component of the circuit. Notice that the inverse circuit will multiply by x^{-1} instead. So, using the same circuit, we can implement both in-place multiplication steps found in [Algorithm 1](#). The remaining multiplication, a modular squaring at Step 10, is out-of-place, and therefore easier to implement.

We explain how to build this circuit from basic modular and non-modular arithmetic circuits. The idea is taken from [14], which optimizes extended Euclidean algorithms (EEA) using *dialog* representations. As we do not use the same representation, we will not use the term “dialog” but the idea remains the same.

3.1 Euclidean Algorithm

We start from a simple, but efficient, implementation of the Euclidean algorithm. The key of this implementation is that, contrary to the usual Extended Euclidean algorithm, it does not construct the Bézout coefficients (r, s) . Instead, it just remembers the choices that were made at each loop iterate, packing them into a *garbage bit-vector*. Then, we can *later* reconstruct (r, s) , gaining space compared to a direct implementation. This is the first key idea from [14].

Algorithm 2 Euclidean Algorithm

Parameters: number of iterations
Input: Integers u, v , u odd
Output: Garbage bit-vector $\mathbf{g}$

```

1: for A fixed number of iterations do
2:    $b_0 := v \bmod 2$ 
3:    $b_1 := u > v$ 
4:   Store  $b_0 \& b_1$  and  $b_0$  in  $\mathbf{g}$ 
5:   if  $b_0 \& b_1$  then
6:      $u, v = v, u$ 
7:   end if
8:   if  $b_0$  then
9:      $v = v - u$ 
10:  end if
11:   $v = v / 2$ 
12: end for

```

▷ At the end, $v = 0$ and $u = 1$

We review the important properties of [Algorithm 2](#).

First, its number of iterations. Let n be the starting bit-size of u and v . Intuitively, each iteration will at least remove one bit of the registers at Step 11, and with probability $1/2$ depending on the value of b_0 , it will remove another bit at Step 9. This means that on average, (uv) is multiplied by $\frac{1}{2} \times \frac{1}{2} + \frac{1}{2} \times \frac{1}{4} = \frac{3}{8}$. So, the average number of iterations should be ℓ such that: $2n - \ell \log_2(8/3) = 0 \implies \ell = \frac{2n}{3 - \log_2 3} \simeq 1.413n$.

Experiments show that this is indeed the case. In addition, the number of iterations follows a normal distribution with standard deviation $\simeq 0.6\sqrt{n}$. In

order to ensure a large success probability on random inputs, we will therefore choose an appropriate constant $c_{\text{iter}} \simeq 2.4$ (four standard deviations) and use a fixed number of $1.413n + c_{\text{iter}}\sqrt{n}$ iterations.

Second, the number of qubits. At the beginning of the algorithm, u and v are stored on $2n$ qubits. Each iteration produces not two, but 1.5 bits of garbage on average. Indeed, we only need to store $b_0 \& b_1$ if $b_0 = 1$. So the total number of output bits that can be expected is $1.5 \times (1.413 + \mathcal{O}(\sqrt{n})) = 2.12n + \mathcal{O}(\sqrt{n})$, and this is the minimal number of qubits with which we can run the algorithm. This situation is very similar to the Jacobi symbol algorithm in [4]: the amount of reduction of (u, v) and the number of garbage bits produced per iterate are the same.

In order to optimize the space, we use *register sharing*. There exists a *padding* amount $c_{\text{pad}}\sqrt{n}$, with $c_{\text{pad}} \simeq 2.3$, such that, with high probability, at each iteration j , the current values of u and v each fit on: $n - 0.5 \log_2(8/3)i + c_{\text{pad}}\sqrt{n}$ bits. This is also experimentally observed. Therefore:

- Operations on u and v are performed on these sub-registers only;
- We use the cleared qubits to store the garbage bits, ensuring a total space in $2.12n + \mathcal{O}(\sqrt{n})$.

The choice of c_{pad} and c_{iter} control the success probability of the algorithm when called on random inputs. According to our experiments, $c_{\text{pad}} = 2.3$ and $c_{\text{iter}} = 2.4$ ensure a success probability large enough so that the point addition circuit succeeds on 10 000 random inputs.

Garbage Encoding. Compared to [4], we modify the storage of garbage bits. Previously we would have used a register in which b_0 and $b_0 \& b_1$ are queued using controlled-shifts. The issue is that the shifts are costly, and also, this makes the total size of the garbage vary, so we would need another $\mathcal{O}(\sqrt{n})$ bits of space to absorb this variation. We use a simpler method here.

Consider three successive iterations producing three pairs of bits. Since these bits are $(b_0, b_0 \& b_1)$ for some b_0, b_1 , they can only be $(0, 0)$, $(1, 0)$ or $(1, 1)$. There are $3^3 = 27 < 2^5$ possible values in input, so we can *compress* this input value into 5 bits, releasing one bit. We define an ad-hoc quantum circuit which does this in-place with 5 Toffoli gates, shown in [Figure 1](#).

With this, our Euclidean algorithm circuit uses exactly $(1.413n + c_{\text{iter}}\sqrt{n})/3 \times 5 \simeq 2.355n + \mathcal{O}(\sqrt{n})$ garbage bits. While this is asymptotically more, the compression step is much more efficient than the previous encoding (accounting at the end for a negligible cost), and the garbage register has now a fixed size.

3.2 Using the Garbage Bit-Vector

As we said above, in the EEA, one also maintains registers (r, s) for the Bézout coefficients which are updated depending on b_0 and $b_0 \& b_1$. If we start the Euclidean algorithm with inputs $(u, v) := (q, x)$, and then run the updates on the Bézout coefficients $(r, s) := (1, 0)$, at the end of the algorithm we have $(r, s) = (0, x^{-1} \bmod q)$.

struction method from [14] is also more efficient because only Algorithm 2 is run twice.

We can now use Algorithm 2 and Algorithm 3 to implement in-place modular multiplication. On input (x, y) , we have three steps:

- We convert x into a garbage bit-vector $\mathbf{g}$ representing the EEA operations;
- We use $\mathbf{g}$ to map y into $xy \bmod q$;
- We convert $\mathbf{g}$ back to x .

The algorithm is Algorithm 4.

Algorithm 4 In-place modular multiplication.

- Input:** Registers x, y
- 1: **Output:** Registers contain $x, yx \bmod q$
- 2: Convert x to bit-vector $\mathbf{g}$ using Algorithm 2 ▷ Registers are now $\mathbf{g}, y$
- 3: Use the Bézout reconstruction algorithm Algorithm 3 on $(y, 0)$ ▷ Registers are now $\mathbf{g}, yx \bmod q$
- 4: Convert back $\mathbf{g}$ to x ▷ Registers are now $x, yx \bmod q$
-

We can make a few observations:

- The space complexity of the entire point addition circuit depends on Algorithm 4 only. All other steps can be implemented using space-efficient components, or sometimes using some trade-offs. For example, ancillas are available during the modular squaring.
- If space-efficient components are used for arithmetic operations, then the dominating step in space complexity is the Bézout reconstruction step. At this point, we need to store $2n$ bits for the two input registers, as well as the bit-vector representation of size $2.355n + \mathcal{O}(\sqrt{n})$. This gives a total space complexity of $4.355n + \mathcal{O}(\sqrt{n})$.
- The GCD construction (Algorithm 2), both forwards and backwards, uses only $2.355n + \mathcal{O}(\sqrt{n})$ qubits in total. With the additional register of n bits stored at this point, this means that around n qubits are available for trade-offs. In particular, we can use more gate-efficient arithmetic circuit like Gidney’s adder [7], which we cannot use during the reconstruction.
- If we still use gate-efficient arithmetic during the reconstruction step, then we will need n additional ancilla qubits, but we will save a few Toffoli gates. This is precisely the difference between the two variants of the Babbush et al.’s circuits [2] as well as ours.

4 Approximate Arithmetic and Modular Arithmetic

At this point, we will obtain a circuit with roughly the same qubit count as [2] (a bit higher), but a larger gate count. We then optimize the arithmetic com-

ponents to reduce the gate count as much as possible. There are three types of optimizations that we perform, which we detail below:

- Using appropriate arithmetic circuits. So far all the circuits used are exact and this has no influence on the success probability.
- Using approximate arithmetic circuits for some steps. This has an influence on the success probability.
- Using approximate modular arithmetic circuits taking advantage of the special form of the prime. This has an influence on the success probability, and makes our circuit dependent on the instance considered.

Trade-offs Between Implementations. When implementing arithmetic circuits with two inputs (adders, comparators, etc.), we commonly have the choice between two implementations:

- Implementations using (almost) no ancillas, based on the CDKM adder [5] or variants. CDKM-based n -bit addition and comparison uses $2n$ Toffoli gates, n -bit controlled addition uses $3n$ Toffoli gates.
- Implementations using around n ancillas using the Gidney adder [7]. This adder uses measurement-based uncomputation to uncompute AND gates efficiently. Addition uses n Toffoli gates, controlled addition uses $2n$ Toffoli gates.

We use strategically components based on the latter when ancillas are available, such as in the modular squaring step, and components based on the former when this is not the case. Notably, during the execution of [Algorithm 2](#) (or the inverse), we have access to roughly n ancillas, so we can use the Gidney adder for the subtraction step. In fact, we do not have *exactly* n ancillas, because the circuit itself is using some of them as padding for the registers u and v . Therefore, we used a hybrid construction as proposed in [7] for the subtraction step in [Algorithm 2](#), which uses exactly the amount of available ancillas.

Approximate Arithmetic Circuits. Shor’s algorithm requires the point addition circuit to work with a large constant probability. This leaves us a lot of freedom to simplify the cost of arithmetic operations. In our circuits, we simplified all comparisons by comparing the most significant bits only (we typically used 40 to 50 bits to ensure a large probability of success). This simplification is used in the comparison step of [Algorithm 2](#) and inside the modular arithmetic circuits.

Approximate Modular Arithmetic. The modular arithmetic circuits that we implement, both for the Bézout coefficient reconstruction ([Algorithm 3](#)) and for the modular squaring step, are:

- (Controlled) Modular addition: $|x, y\rangle \mapsto |x, y + x \bmod q\rangle$
- Modular doubling: $|x\rangle \mapsto |2x \bmod q\rangle$.

These circuits only need to work when their inputs are drawn uniformly at random, with a single exception. During the Bézout reconstruction ([Algorithm 3](#)), for the first $c_{\text{iter}}\sqrt{n}$ iterations, we have to handle the case where $x + y = q$. This should not appear if we pick the inputs at random, but it does appear in our case for the first empty iterations of the algorithm.

Our starting points for these two circuits are taken from [20]. We start by simplifying the comparisons as shown above, then we simplify them further using the special form of the prime q . In the following, we give the circuits as pseudocode which is similar to our python code. The commands used in pseudocode are: `input` to allocate an input register (which is also output), of a given type, `anc` to allocate ancillas, basic gates, operations on registers such as `shift_right`, `sub` (subtraction), `cadd` (controlled addition), `lt` (lower than). All these operations are in-place, and `lt` XORs the result of the comparison to its last input register. `clt` is the controlled version. We use “+” to denote concatenation of registers, seen as lists of qubits.

Doubling. On input x , the doubling circuit of [20] does a shift, a subtraction by q , and a controlled addition by q . This is given in [Algorithm 5](#).

Algorithm 5 Exact modular doubling circuit from [20].

```
x_reg = input(ModIntType(q))
anc1, anc2 = anc(1), anc(1)
shift_right(x_reg + anc1, 1)
sub_uint(q, x_reg + anc1 + anc2)
cadd_uint(anc2, q, x_reg + anc1)
x(anc2)
cx(x_reg[0], anc2)
```

Our “efficient” modular doubling, which works for any prime, is given in [Algorithm 6](#). The only non-exact step is the comparison, where we take the most significant bits of the current register and q , whereas the exact version of this circuit would have compared the entire numbers. We notice that the cost of modular doubling has been reduced to approximately one n -bit controlled constant addition.

When q is a pseudo-Mersenne prime of the form $2^u - f$, where $f \ll 2^u$, we can further simplify this circuit. Indeed, we can further approximate the comparison by just checking whether the MSB is 1, and in that case, the controlled-subtraction by q becomes an erasure of the MSB, followed by adding f into the register x . Since f is small, we only need to add it into the LSBs (since carries will only propagate a certain amount). The exact range determines the success probability of the algorithm. This is given in [Algorithm 7](#).

Addition. On input x, y , the controlled modular addition circuit of [20] does a controlled addition, followed by a subtraction of q , a controlled addition of q

Algorithm 6 Non-exact modular doubling circuit.

```

x_reg = input(ModIntType(q))
anc1, anc2 = anc(1), anc(1)
shift_right(x_reg + anc1, 1)
lt_uint(x_reg[-padding:] + anc1, q_msbs, anc2) # non-exact step
self.x(anc2)
csub_uint(anc2, q, x_reg + anc1)
cx(x_reg[0], anc2)

```

Algorithm 7 Pseudo-Mersenne modular doubling circuit.

```

x_reg = input(ModIntType(q))
anc1 = anc(1)
shift_right(x_reg + anc1, 1)
cadd_uint(anc1, f, x_reg[:lsbs])
cx(x_reg[0], anc1[0])

```

and a comparison to erase an ancilla qubit. (Note that compared to the non-controlled circuit, the subtraction / addition of q do not need to be modified).

Algorithm 8 Exact controlled modular addition circuit from [20].

```

control = input(BoolType())
x_reg = input(ModIntType(p))
y_reg = input(ModIntType(p))
anc_x = anc(1)
anc_y = anc(1)
cadd_uint(control, x_reg + anc_x, y_reg + anc_y)
sub_uint(p, y_reg + anc_y)
cadd_uint(anc_y, p, y_reg)
clt_uint(control, y_reg, x_reg, anc_y)
x(anc_y)

```

Our first simplification is given in [Algorithm 9](#). We first perform a non-modular (controlled) addition, then check if the result overflows (but only on the MSBs). If it does, we subtract q from the result. The ancilla bit checking the overflow can then be erased by the condition $y < x$, since the only case where this happens is precisely when there was an overflow. Both comparisons being performed only on the MSBs, the remaining dominating operations are a controlled addition and a controlled subtraction by a constant.

When q is a pseudo-Mersenne prime of the form $2^u - f$, where $f \ll 2^u$, we can further simplify this circuit. Here we need two variants: one that handles

Algorithm 9 Non-exact controlled modular addition circuit.

```

control = input(BoolType())
x_reg = input(ModIntType(p))
y_reg = input(ModIntType(p))
anc_x = anc(1)
anc_y = anc(1)
cadd_uint(control, x_reg + anc_x, y_reg + anc_y)
anc = anc(1)
lt_uint(y_reg[-msbs:] + anc_y, q_msbs, anc)
self.x(anc[0])
csub_uint(anc, q, y_reg + anc_y)
clt_uint(control, y_reg[-msbs:], x_reg[-msbs:], anc)

```

the case $x + y = q$ (which would not happen randomly), and one that does not need to.

Algorithm 10 Pseudo-Mersenne controlled modular addition circuit, which does not handle $x + y = q$.

```

control = input(BoolType())
x_reg = input(ModIntType(q))
y_reg = input(ModIntType(q))
anc_x = anc(1)
anc_y = anc(1)
cadd_uint(control, x_reg + anc_x, y_reg + anc_y)
cadd_uint(anc_y, f, y_reg[:lsbs])
clt_uint(control, y_reg[-msbs:], x_reg[-msbs:], anc_y)

```

The latter is given in [Algorithm 10](#). We first perform the controlled addition, then if the register overflows, we add f in the LSBs of the output register y . We erase the overflow bit by a new comparison.

The algorithm handling the case $x + y = q$ is given in [Algorithm 11](#). It detects whether $x + y = q$ by checking if all MSBs are 1. If $x + y = q$, then after reduction, the register y should contain 0. Thus the bit is erased by checking if all MSBs are 0.

Difference of Cost. As can be seen in [Table 1](#), there is a significant difference between the case of a pseudo-Mersenne prime as in `secp256k1` and the generic case. This comes entirely from the different implementations given above. An important factor is that, for the operations which happen during the Bézout reconstruction step, we need to optimize the space, so when we need to add or subtract q , we use Gidney’s constant adder [8] with dirty ancillas (which costs $3n$ Toffoli gates for n -bit integers). Reducing the constant addition steps from q to f is a significant gain of gate count.

Algorithm 11 Pseudo-Mersenne controlled modular addition circuit which handles $x + y = q$.

```

control = input(BoolType())
x_reg = input(ModIntType(q))
y_reg = input(ModIntType(q))
anc_x = anc(1)
anc_y = anc(1)
cadd_uint(control, x_reg + anc_x, y_reg + anc_y)
anc = anc(1)
y_is_q = all(y_reg[-msbs:])
cxor(y_is_q, q, y_reg) # Controlled-XOR of constant (CNOT gates)
cx(anc_y, anc)
cadd_uint(anc, f, y_reg[:lsbs])
cx(anc, anc_y)
clt_uint(control, y_reg[-msbs:], x_reg[-msbs:], anc)
cx(y_is_q, anc)
y_is_q = all(not y_reg[-msbs:])

```

5 Discussion

After constructing the entire circuit, it is interesting to look at the proportion of the CCX cost of all its sub-circuits (we count together CCX and And gates resulting from Gidney’s adder construction [7]). Our estimations are given in [Table 3](#). We can see as noticed above that the difference in gate count between a pseudo-Mersenne prime and a generic prime can be attributed to the modular addition and doubling circuits, and more precisely to the constant additions therein.

Table 3. Proportion of the CCX cost of different sub-circuit components.

	Space-optimized for <code>secp256k1</code>	Space-optimized for any prime
Main components:		
Modular squaring	9 %	10 %
In-place multiplier and its inverse	90 %	90 %
Bézout reconstruction	54 %	64 %
GCD construction	36 %	24 %
Some sub-components:		
Controlled-swap	19 %	12 %
Hybrid adder	22 %	14 %
Controlled modular adder (and inverse)	39 %	45 %
Modular double (and inverse)	8 %	24 %
Gidney constant adder (controlled and not)	15 %	34 %

Acknowledgments. The author thanks Pierre-Alain Fouque, Dahmun Goudarzi, Amaury Pouly, and Yixin Shen for motivating discussions. This work has been supported by the French Agence Nationale de la Recherche through the France 2030 program under grant agreement No. ANR-22-PETQ-0008 PQ-TLS

References

1. Babbush, R., Gidney, C., Berry, D.W., Wiebe, N., McClean, J., Paler, A., Fowler, A., Neven, H.: Encoding electronic spectra in quantum circuits with linear T complexity. *Physical Review X* **8**(4), 041015 (2018). <https://doi.org/10.1103/PhysRevX.8.041015>
2. Babbush, R., Zalcman, A., Gidney, C., Broughton, M., Khattar, T., Neven, H., Bergamaschi, T., Drake, J., Boneh, D.: Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations. arXiv preprint arXiv:2603.28846 (2026)
3. Cheignard, C., Fouque, P., Schrottenloher, A.: Reducing the number of qubits in quantum factoring. In: CRYPTO (2). Lecture Notes in Computer Science, vol. 16001, pp. 384–415. Springer (2025). https://doi.org/10.1007/978-3-032-01878-6_13
4. Cheignard, C., Fouque, P.A., Schrottenloher, A.: Reducing the number of qubits in quantum discrete logarithms on elliptic curves. EUROCRYPT 2026 (2026), <https://eprint.iacr.org/2026/280>
5. Cuccaro, S.A., Draper, T.G., Kutin, S.A., Moulton, D.P.: A new quantum ripple-carry addition circuit. arXiv preprint quant-ph/0410184 (2004)
6. Ekerå, M.: Revisiting shor’s quantum algorithm for computing general discrete logarithms. CoRR **abs/1905.09084** (2019), <http://arxiv.org/abs/1905.09084>
7. Gidney, C.: Halving the cost of quantum addition. *Quantum* **2**, 74 (2018). <https://doi.org/10.22331/Q-2018-06-18-74>, <https://doi.org/10.22331/q-2018-06-18-74>
8. Gidney, C.: A classical-quantum adder with constant workspace and linear gates. arXiv preprint arXiv:2507.23079 (2025)
9. Gidney, C.: How to factor 2048 bit RSA integers with less than a million noisy qubits. arXiv preprint arXiv:2505.15917 (2025)
10. Gidney, C., Ekerå, M.: How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum* **5**, 433 (2021). <https://doi.org/10.22331/q-2021-04-15-433>
11. Gouzien, É., Ruiz, D., Le Régent, F.M., Guillaud, J., Sangouard, N.: Performance analysis of a repetition cat code architecture: Computing 256-bit elliptic curve logarithm in 9 hours with 126 133 cat qubits. *Physical review letters* **131**(4), 040602 (2023)
12. Griffiths, R.B., Niu, C.S.: Semiclassical fourier transform for quantum computation. *Physical Review Letters* **76**(17), 3228 (1996). <https://doi.org/10.1103/PhysRevLett.76.3228>
13. Häner, T., Jaques, S., Naehrig, M., Roetteler, M., Soeken, M.: Improved quantum circuits for elliptic curve discrete logarithms. In: PQCrypto. Lecture Notes in Computer Science, vol. 12100, pp. 425–444. Springer (2020). https://doi.org/10.1007/978-3-030-44223-1_23
14. Khattar, T., Shutty, N., Gidney, C., Zalcman, A., Yosri, N., Maslov, D., Babbush, R., Jordan, S.P.: Verifiable quantum advantage via optimized DQI circuits. arXiv preprint arXiv:2510.10967 (2025)

15. Litinski, D.: How to compute a 256-bit elliptic curve private key with only 50 million toffoli gates. arXiv preprint arXiv:2306.08585 (2023)
16. National Institute of Standards and Technology: Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography. Tech. Rep. SP 800-56A Rev. 3, National Institute of Standards and Technology (Apr 2018). <https://doi.org/10.6028/NIST.SP.800-56Ar3>, <https://doi.org/10.6028/NIST.SP.800-56Ar3>
17. Nielsen, M.A., Chuang, I.: Quantum computation and quantum information (2002)
18. NIST: Submission requirements and evaluation criteria for the post-quantum cryptography standardization process (2016), <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf>
19. Proos, J., Zalka, C.: Shor’s discrete logarithm quantum algorithm for elliptic curves. Quantum Inf. Comput. **3**(4), 317–344 (2003). <https://doi.org/10.26421/QIC3.4-3>, <https://doi.org/10.26421/QIC3.4-3>
20. Roetteler, M., Naehrig, M., Svore, K.M., Lauter, K.: Quantum resource estimates for computing elliptic curve discrete logarithms. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 241–270. Springer (2017)
21. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM J. Comput. **26**(5), 1484–1509 (1997). <https://doi.org/10.1137/S0097539795293172>